

Quantization Techniques for Model Compression

.By Ra'ed Ismail

Introduction:

Quantization is a model compression technique that reduces the memory usage and computational cost of machine learning models by representing model parameters and computations using lower-precision numerical formats. Instead of using high-precision values such as 32-bit floating-point numbers (FP32), a model can use 16-bit, 8-bit, or even lower-precision representations.

Quantization is especially useful when deploying large deep learning models on devices with limited memory or computational resources.

Why Use Quantization:

Smaller model size: Lower-precision weights require less storage.

Lower memory usage: The model requires less RAM or VRAM during inference.

Faster inference: Many CPUs, GPUs, and specialized hardware can perform low-precision operations efficiently.

Lower power consumption: Fewer and simpler numerical operations can reduce energy usage.

Easier deployment: Smaller models are more suitable for mobile devices, edge devices, and resource-limited environments.

For example, converting a model's weights from FP32 to INT8 can reduce the storage required for the weights by approximately **75%**, because each value uses 8 bits instead of 32 bits.

Main Quantization Techniques:

1. Post-Training Quantization (PTQ): Post-Training Quantization is applied after the model has already been trained. The original model is converted from a higher-precision format to a lower-precision format without retraining the model.

Common approaches include:

Dynamic Range Quantization: Weights are quantized, while some activations are converted dynamically during inference.

Static Quantization: Both weights and activations are quantized using calibration data.

Weight-Only Quantization: Only the model weights are quantized while activations remain in higher precision.

2. Quantization-Aware Training (QAT): Quantization-Aware Training simulates the effects of quantization during training. The model learns to maintain good performance despite using lower-precision values.

QAT can usually preserve accuracy better than simple post-training quantization, especially when a model is highly sensitive to reduced numerical precision. However it requires additional training time and computational resources.

3. Weight Quantization: Weight quantization reduces the precision of the model's learned parameters.

For example: FP32 $\rightarrow$ INT8, A 32-bit floating-point weight is represented using an 8-bit integer.

This technique is particularly important for large language models, where the model weights can occupy several gigabytes of memory.

4. Activation Quantization:

Activation quantization reduces the precision of the intermediate values produced by neural network layers.

For example: FP32 activations $\rightarrow$ INT8 activations

Quantizing both weights and activations can provide significant memory and inference-speed improvements, but it may introduce a larger accuracy loss if the model is sensitive to quantization.

Conclusion:

Quantization is an important technique for deploying machine learning models efficiently. It reduces model size, memory consumption, and computational requirements by using fewer bits to represent model parameters and computations. Post-Training Quantization is convenient when a trained model is already available, while Quantization-Aware Training can provide better accuracy when the model requires additional adaptation. The appropriate technique depends on the model, available hardware, and acceptable accuracy trade-offs.

